VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
DINISHRA RAWAT	24BCE10181
Slot	**Date of Submission**
E11	31/03/2026

1. Introduction

Open source software refers to software whose source code is freely available for anyone to view, modify, and distribute. It plays a crucial role in modern computing, as most systems and applications depend on open-source technologies. Today, open-source software forms the backbone of the internet, operating systems, mobile devices, and cloud computing platforms. It encourages collaboration among developers across the world and promotes innovation by allowing people to improve existing solutions instead of building everything from scratch.

This project focuses on analyzing one such open-source tool, Git. Git is a widely used version control system that helps developers track changes in their code and collaborate efficiently. The goal of this project is to understand not only the functionality of Git but also its origin, philosophy, licensing model, and its impact on the software industry. By studying Git, we gain insight into how open-source projects are developed and maintained.

Additionally, this project provides hands-on experience with Linux systems and basic shell scripting. This practical exposure helps in understanding how open-source tools work in real environments, making the learning process more effective and meaningful.

2. ABOUT GIT

Git is a distributed version control system used to track changes in source code during software development. It allows multiple developers to work on the same project efficiently without conflicts by maintaining a complete history of changes. Unlike traditional systems, Git does not rely on a central server; instead, every developer has a full copy of the project repository. This makes it faster, more reliable, and less dependent on internet connectivity.

One of the key advantages of Git is its ability to manage different versions of a project through features like branching and merging. Developers can create separate branches to work on new features or fix bugs without affecting the main code. Once the changes are tested, they can be merged back into the main project smoothly. This flexibility makes Git highly suitable for both small and large-scale development projects.

Git is widely used in the software industry and forms the backbone of platforms like GitHub and GitLab. These platforms provide additional features such as collaboration, code review, and project management, making teamwork more organized and efficient. Due to its speed, flexibility, and strong community support, Git has become an essential tool for developers. It not only improves productivity but also ensures better control and security over the code, making it a fundamental part of modern software development.

3. PART A — ORIGIN AND PHILOSOPHY

Git was created in 2005 by Linus Torvalds, who is also known for developing the Linux operating system. Before Git came into existence, the development of the Linux kernel relied on a proprietary version control system called BitKeeper. Although BitKeeper was efficient and widely used at that time, it was not open source and came with certain licensing restrictions. Initially, the Linux development community was allowed to use it for free, but conflicts arose between the developers and the company that owned BitKeeper. As a result, access to the tool was revoked, leaving the Linux developers without a proper system to manage their code.

This situation created an urgent need for a new version control system that would be both powerful and free from such restrictions. Linus Torvalds decided to take matters into his own hands and develop a tool that could meet the needs of large-scale collaborative development. His goal was to design a system that was fast, efficient, reliable, and capable of handling contributions from thousands of developers working simultaneously on different parts of the project.

Git was built with a distributed approach, meaning that every developer has a complete copy of the entire codebase along with its history. This was a major improvement over centralized systems, as it allowed developers to work independently without constantly relying on a central server. It also improved performance and reduced the risk of data loss. Another important feature of Git is its strong focus on data integrity. Every change made to the code is tracked and verified, ensuring that the project history remains accurate and secure.

In addition to solving the immediate problem faced by Linux developers, Git was also released as open-source software. This decision allowed anyone to use, modify, and contribute to the tool, which led to its rapid adoption across the software industry. Over time, Git became one of the most widely used version control systems in the world, forming the foundation of modern software development practices. Its creation not only solved a technical problem but also reinforced the importance of open-source principles such as collaboration, transparency, and freedom.Git was created in 2005 by **Linus Torvalds**, who is also known for developing the Linux operating system. Before Git came into existence, the development of the Linux kernel relied on a proprietary version control system called BitKeeper. Although BitKeeper was efficient and widely used at that time, it was not open source and came with certain licensing restrictions. Initially, the Linux development community was allowed to use it for free, but conflicts arose between the developers and the company that owned BitKeeper. As a result, access to the tool was revoked, leaving the Linux developers without a proper system to manage their code.

This situation created an urgent need for a new version control system that would be both powerful and free from such restrictions. Linus Torvalds decided to take matters into his own hands and develop a tool that could meet the needs of large-scale collaborative development. His goal was to design a system that was fast, efficient, reliable, and capable of handling contributions from thousands of developers working simultaneously on different parts of the project.

Git was built with a distributed approach, meaning that every developer has a complete copy of the entire codebase along with its history. This was a major improvement over centralized systems, as it allowed developers to work independently without constantly relying on a central server. It also improved performance and reduced the risk of data loss. Another important feature of Git is its strong focus on data integrity. Every change made to

the code is tracked and verified, ensuring that the project history remains accurate and secure.

In addition to solving the immediate problem faced by Linux developers, Git was also released as open-source software. This decision allowed anyone to use, modify, and contribute to the tool, which led to its rapid adoption across the software industry. Over time, Git became one of the most widely used version control systems in the world, forming the foundation of modern software development practices. Its creation not only solved a technical problem but also reinforced the importance of open-source principles such as collaboration, transparency, and freedom.

A2: License (GPL v2)

Git is licensed under the GNU General Public License version 2 (GPL v2), which is one of the most widely used open-source licenses in the world. This license ensures that Git remains free and accessible to everyone. Under GPL v2, users are allowed to freely use the software for any purpose, study how it works, modify it according to their needs, and distribute copies to others. This makes Git not just a tool, but a shared resource that anyone can improve and benefit from.

The concept of free software under this license is based on four essential freedoms. The first is the freedom to run the program for any purpose, without restrictions. The second is the freedom to study the source code and understand how the software works. The third is the freedom to modify the program and adapt it to specific requirements. Finally, the fourth freedom allows users to distribute copies of the original or modified software to others. These freedoms promote collaboration, learning, and innovation within the developer community.

However, GPL v2 also includes an important condition known as “copyleft.” This means that if someone modifies Git and distributes the modified version, they must also release their changes under the same GPL v2 license. In simple terms, any improvements made to the software must remain open and accessible to others. This ensures that no one can take the code, make it proprietary, and restrict access to it.

Overall, the GPL v2 license plays a crucial role in maintaining the openness and fairness of Git. It protects the rights of users while encouraging developers to contribute back to the community. This licensing model has been a key factor in Git’s success, as it builds trust and promotes a culture of sharing and continuous improvement.

A3: Ethics of Open Source

Open source software promotes transparency, collaboration, and innovation. Since the source code is openly available, anyone can examine how the software works, identify bugs, and suggest improvements. This level of openness builds trust among users and developers because there is nothing hidden. It also creates a strong learning environment, especially for students and beginners, as they can study real-world code and understand how complex systems are built. By allowing developers from different parts of the world to contribute, open source encourages collaboration and leads to faster development and better-quality software.

Another important advantage of open source is that it reduces duplication of effort. Instead of building everything from scratch, developers can use existing tools and improve them further. This not only saves time but also drives innovation, as people can focus on adding new features rather than repeating basic work. Many successful technologies today, including operating systems, programming languages, and web platforms, are built on open-source foundations.

However, there are also debates and concerns related to open source. One major question is whether it is fair for companies to make large profits using open-source software without contributing back to the community. While some companies actively support open source by contributing code, funding projects, or maintaining tools, others may simply use the software for commercial gain. This raises concerns about sustainability, as open-source projects often rely on volunteers who may not receive direct financial benefits for their work.

In my opinion, open source is extremely beneficial for learning, innovation, and technological growth. It creates opportunities for collaboration and allows knowledge to be shared freely. However, for the ecosystem to remain healthy and sustainable, companies that benefit from open-source software should also give back in some form. This could be through contributions, financial support, or active participation in the community. In this way, open source can continue to grow and benefit everyone in the long term.

Part B — Linux Footprint

1. Installation of Git

Git was installed on the Linux system using the Advanced Package Tool (APT), which is commonly used in Ubuntu-based distributions. The installation process was simple and required only a few commands.

The following command was used to install Git:

sudo apt update
sudo apt install git

The first command updates the package list, while the second command installs Git from the official repositories. This ensures that the latest stable version of Git is installed on the system.

```
Get:1 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Hit:2 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:3 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:4 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [3067 kB]
Get:5 https://download.docker.com/linux/ubuntu jammy InRelease [48.5 kB]
Get:6 https://download.docker.com/linux/ubuntu jammy/stable amd64 Packages [72.8 kB]
Get:7 http://security.ubuntu.com/ubuntu jammy-security/main Translation-en [437 kB]
Get:8 http://security.ubuntu.com/ubuntu jammy-security/main amd64 c-n-f Metadata [14.1 kB]
Get:9 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages [5356 kB]
Get:10 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3328 kB]
Get:12 http://security.ubuntu.com/ubuntu jammy-security/restricted Translation-en [1025 kB]
Get:13 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 c-n-f Metadata [680 B]
Get:14 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1020 kB]
Get:15 http://archive.ubuntu.com/ubuntu jammy-updates/main Translation-en [508 kB]
Get:16 http://security.ubuntu.com/ubuntu jammy-security/universe Translation-en [226 kB]
Get:17 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 c-n-f Metadata [22.7 kB]
Get:18 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 c-n-f Metadata [19.5 kB]
Get:19 http://archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [5518 kB]
Get:20 http://archive.ubuntu.com/ubuntu jammy-updates/restricted Translation-en [1053 kB]
Get:21 http://archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 c-n-f Metadata [676 B]
Get:22 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1258 kB]
Get:23 http://archive.ubuntu.com/ubuntu jammy-updates/universe Translation-en [315 kB]
Get:24 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 c-n-f Metadata [30.4 kB]
Get:25 http://archive.ubuntu.com/ubuntu jammy-backports/main amd64 Packages [69.4 kB]
Get:26 http://archive.ubuntu.com/ubuntu jammy-backports/main amd64 c-n-f Metadata [412 B]
Get:27 http://archive.ubuntu.com/ubuntu jammy-backports/universe amd64 Packages [30.4 kB]
Get:28 http://archive.ubuntu.com/ubuntu jammy-backports/universe Translation-en [16.9 kB]
Get:29 http://archive.ubuntu.com/ubuntu jammy-backports/universe amd64 c-n-f Metadata [672 B]
Fetched 23.8 MB in 30s (790 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
212 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Suggested packages:
  git-daemon-run | git-daemon-sysvinit git-doc git-email git-gui gitk
  gitweb git-cvs git-mediawiki git-svn
The following packages will be upgraded:
  git
1 upgraded, 0 newly installed, 0 to remove and 211 not upgraded.
Need to get 3174 kB of archives.
After this operation, 0 B of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 git amd64 1:2.34.1-1ubuntu1.17 [3174 kB]
Fetched 3174 kB in 4s (735 kB/s)
(Reading database ... 34480 files and directories currently installed.)
Preparing to unpack .../git_1%3a2.34.1-1ubuntu1.17_amd64.deb ...
Unpacking git (1:2.34.1-1ubuntu1.17) over (1:2.34.1-1ubuntu1.11) ...
Setting up git (1:2.34.1-1ubuntu1.17) ...
```

2. Verification of Installation

After installation, Git was verified using the following command:

git --version

This command displays the currently installed version of Git, confirming that the installation was successful.

```
@DESKTOP-R8K2IV5:~$ git --version
git version 2.34.1
```

3. Location of Git in Linux

To find where Git is installed in the system, the following command was used:

which git

The output showed that Git is located in the `/usr/bin` directory. This directory stores executable binary files that are accessible system-wide.

```
@DESKTOP-R8K2IV5:~$ which git
/usr/bin/git
```

4. Configuration Files

Git stores its configuration settings in configuration files. The main configuration file for a user is located at:

~/.gitconfig

This file contains user-specific settings such as username, email, and preferences. There is also a system-wide configuration file located at:

/etc/gitconfig

These configuration files allow users to customize how Git behaves on their system.

5. Logs and Data Storage

Git does not maintain traditional system log files like other services. Instead, it stores logs and history inside each repository.

When a Git repository is created, a hidden folder named `.git` is generated. Inside this folder, all the logs, commit history, and version information are stored.

Example:

project-folder/.git/logs

This ensures that all changes are tracked and can be restored if needed.

6. User and Permissions

Git runs under the currently logged-in user and does not require a separate system user. This makes it simple to use and manage.

Since Git does not run as a background service, it only operates when commands are executed by the user. This improves security and reduces unnecessary system resource usage.

7. Service Management

Unlike web servers or databases, Git is not a service that runs continuously in the background. Therefore, it does not require commands like start, stop, or restart.

Instead, Git works as a command-line tool that executes only when needed.

8. Update Mechanism

Git updates are managed through the Linux package manager. When system updates are performed, Git can also be updated using:

sudo apt upgrade

This ensures that the system always has the latest security patches and improvements provided by the open-source community.

Part C — The FOSS Ecosystem

Git does not exist as a standalone tool; it is part of a large and interconnected open-source ecosystem. It works together with many other tools, technologies, and communities that support modern software development. This ecosystem plays a crucial role in making Git powerful, flexible, and widely adopted across the world.

One of the key aspects of Git's ecosystem is its dependencies and underlying technologies. Git is primarily written in the C programming language, which makes it fast and efficient. It relies on standard system libraries and tools available in Linux environments. These dependencies ensure that Git can run smoothly on different platforms while maintaining high performance.

Git has also enabled the development of many platforms and tools built on top of it. Popular platforms like GitHub and GitLab use Git as their core system to provide additional features such as remote repositories, collaboration tools, issue tracking, and continuous integration/continuous deployment (CI/CD) pipelines. These platforms allow developers to collaborate from different parts of the world, making teamwork more organized and efficient.

In addition to platforms, many development tools and workflows depend on Git. For example, DevOps practices heavily rely on Git for version control, automation, and deployment. Tools used for building, testing, and deploying applications often integrate directly with Git repositories.

Another important part of the ecosystem is the open-source community. Git has a large and active community of contributors who continuously improve the software. Developers from around the world contribute code, fix bugs, and suggest new features. The project is maintained through collaboration, discussions, and contributions, ensuring its continuous growth and reliability.

Overall, Git plays a central role in the open-source ecosystem. It not only depends on other tools but also enables countless technologies and workflows. Its strong community support and widespread usage make it a fundamental component of modern software development.

Part D — Open Source vs Proprietary (Critical Analysis)

Dimension	Open Source	Proprietary Alternative
Cost	Free to use	Often paid or licensed
Security (who can audit the code?)	Can be audited by anyone	Depends on company
Performance	Fast (distributed system)	Slower (centralized system)
Freedom to modify	Highly customizable	Limited customization
Community vs corporate control	Community-driven	Company-controlled
Your overall verdict	Overall better	Less better

Explanation

Git, being an open-source software, provides complete freedom to users. Anyone can download it, use it, modify it, and even distribute it without paying any cost. This makes it highly accessible, especially for students, developers, and small organizations. Since the source code is open, developers can inspect it for security issues and improve it continuously.

On the other hand, proprietary systems like SVN (in some enterprise use cases) or other paid tools are controlled by companies. Users do not have access to the source code, which limits transparency. While such systems may provide dedicated support and structured services, they often lack flexibility and freedom compared to open-source tools.

Another major difference is the working model. Git uses a distributed system where every user has a complete copy of the repository. This makes it faster and more reliable. In contrast, centralized systems depend on a single server, which can become a point of failure.

🟩 Final Verdict

In conclusion, Git is a more powerful and flexible option compared to proprietary alternatives. Its open-source nature allows continuous improvement, strong community support, and wide adoption across the industry. While proprietary tools may offer structured support, Git provides greater freedom, performance, and scalability.

Based on this analysis, Git is highly recommended for real-world development. It not only improves productivity but also supports collaboration and innovation. I would also be interested in contributing to Git or similar open-source projects in the future, as it promotes learning and community-driven development.

4. Shell Script Tasks

Script 1 — System Identity Report

```
# Script 1: System Identity Report

SOFTWARE_CHOICE="Git"

KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DATE=$(date)
DISTRO=$(lsb_release -d | cut -f2)

echo "================================"
echo " Open Source Audit - $STUDENT_NAME"
echo "================================"
echo "Software: $SOFTWARE_CHOICE"
echo "Kernel  : $KERNEL"
echo "User    : $USER_NAME"
echo "Uptime  : $UPTIME"
echo "Date    : $DATE"
echo "Distro  : $DISTRO"
echo "License : GPL (Linux Kernel)"
================================
 Open Source Audit
================================
Software: Git
Kernel  : 6.6.87.2-microsoft-standard-WSL2
User    : aditi_39
Uptime  : up 31 minutes
Date    : Thu Mar 26 12:09:38 IST 2026
Distro  : Ubuntu 22.04.3 LTS
License : GPL (Linux Kernel)
```

This script is designed to display basic information about the Linux system in a structured and readable format. It acts like a system identity report, providing important details about the environment in which the system is running.

The script begins by defining variables such as the student name and the chosen software, which in this case is Git. It then uses command substitution to collect system information. The `uname -r` command is used to get the kernel version, while the `whoami` command identifies the current logged-in user. The system uptime is obtained using the

`uptime -p` command, and the current date and time are retrieved using the `date` command.

To identify the Linux distribution, the script uses the `lsb_release -d` command and extracts the relevant part using the `cut` command. All this information is stored in variables and later displayed using the `echo` command in a formatted manner.

The script also includes a message indicating that the Linux kernel is licensed under the GNU General Public License (GPL), highlighting its open-source nature.

Overall, this script demonstrates the use of variables, command substitution, and output formatting in shell scripting. It provides a simple but effective way to display system information, which can be useful for system monitoring and reporting purposes.

Script 2 — FOSS Package Inspector

```
PACKAGE="git"

if dpkg -l | grep -q $PACKAGE; then
    echo "$PACKAGE is installed."
    dpkg -s $PACKAGE | grep -E 'Version|Maintainer|Description'
else
    echo "$PACKAGE is NOT installed."
fi

case $PACKAGE in
    git) echo "Git: distributed version control system" ;;
    apache2) echo "Apache: web server" ;;
    mysql-server) echo "MySQL: database system" ;;
    vlc) echo "VLC: media player" ;;
esac
git is installed.
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Version: 1:2.34.1-1ubuntu1.17
Description: fast, scalable, distributed revision control system
Original-Maintainer: Jonathan Nieder <jrnieder@gmail.com>
Git: distributed version control system
```

This script is designed to check whether a specific open-source software package is installed on the system and display its details. In this case, the script checks for the Git package.

The script begins by storing the package name in a variable called `PACKAGE`. It then uses an `if-else` condition to determine whether the package is installed. The command `dpkg -l` lists all installed packages, and it is combined with `grep` to search for the specified package name. If the package is found, the script prints a message confirming that it is installed.

Next, the script uses the command `dpkg -s` to display detailed information about the package, such as its version, maintainer, and description. The output is filtered using `grep` to show only relevant details, making it more readable.

If the package is not installed, the script prints a message indicating that it is not available on the system.

Finally, a `case` statement is used to print a short description of the package based on its name. For Git, it displays that it is a distributed version control system.

This script demonstrates the use of important shell scripting concepts such as variables, conditional statements (`if-else`), command-line utilities, and `case` statements. It is useful for quickly checking the status and details of software packages in a Linux system.

Script 3 — Disk and Permission Auditor

```
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        echo "$DIR => $PERMS | Size: $SIZE"
    fi
done

# Git config directory
CONFIG="$HOME/.gitconfig"

if [ -f "$CONFIG" ]; then
    ls -l $CONFIG
else
    echo "Git config not found"
fi
Directory Audit Report
/etc => drwxr-xr-x root root | Size: 4.1M
/var/log => drwxr-xr-x root syslog | Size: 277M
/home => drwxr-xr-x root root | Size: 1.1G
/usr/bin => drwxr-xr-x root root | Size: 375M
/tmp => drwxrwxrwt root root | Size: 16K
```

This script is designed to analyze important system directories and display their disk usage along with their permissions and ownership details. It helps in understanding how storage is used and how access is controlled in a Linux system.

The script begins by defining an array named `DIRS`, which contains a list of commonly used system directories such as `/etc`, `/var/log`, `/home`, `/usr/bin`, and `/tmp`. These directories are important because they store configuration files, logs, user data, system binaries, and temporary files.

A `for` loop is used to iterate through each directory in the list. For every directory, the script first checks whether it exists using an `if` condition. This ensures that the script does not produce errors if a directory is missing.

If the directory exists, the script uses the `ls -ld` command to retrieve its permissions,

owner, and group information. The output is processed using the `awk` command to extract only the relevant fields. To calculate disk usage, the script uses the `du -sh` command, which provides a human-readable size of the directory.

The script then displays the directory name along with its permissions and size in a clear and structured format.

In addition, the script checks for the existence of the Git configuration file located at `~/.gitconfig`. If the file exists, it displays its permissions using the `ls -l` command. This helps in verifying whether the configuration file is accessible and properly secured.

This script demonstrates important shell scripting concepts such as arrays, loops, conditional statements, and command-line utilities. It is useful for system administrators to monitor disk usage and ensure proper permission settings in the system.

Script 4 — Log File Analyzer

```
  GNU nano 6.2
#!/bin/bash

LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0

if [ -z "$LOGFILE" ]; then
    echo "Usage: ./script4.sh <logfile> [keyword]"
    exit 1
fi

if [ ! -f "$LOGFILE" ]; then
    echo "File not found"
    exit 1
fi

while read LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT+1))
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times"

echo "Last 5 matches:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

```
Keyword 'error' found 98 times
Last 5 matches:
Mar 26 12:27:29 DESKTOP-R8K2IV5 containerd[250]: time="2026-03-26T12:27:29.175786100+05:30" level=info msg="skip loading plugin \"io.containerd.snapshotter.
v1.zfs\"..." error="path /var/lib/containerd/io.containerd.snapshotter.v1.zfs must be a zfs filesystem to be used with the zfs snapshotter: skip plugin" typ
e=io.containerd.snapshotter.v1
Mar 26 12:27:29 DESKTOP-R8K2IV5 containerd[250]: time="2026-03-26T12:27:29.182865500+05:30" level=info msg="skip loading plugin \"io.containerd.tracing.proc
essor.v1.otlp\"..." error="skip plugin: tracing endpoint not configured" type=io.containerd.tracing.processor.v1
Mar 26 12:27:29 DESKTOP-R8K2IV5 containerd[250]: time="2026-03-26T12:27:29.182924200+05:30" level=info msg="skip loading plugin \"io.containerd.internal.v1.
tracing\"..." error="skip plugin: tracing endpoint not configured" type=io.containerd.internal.v1
Mar 26 12:27:30 DESKTOP-R8K2IV5 kernel: [    6.567010] WSL (216) ERROR: CheckConnection: getaddrinfo() failed: -5
Mar 26 12:27:31 DESKTOP-R8K2IV5 dockerd[306]: time="2026-03-26T12:27:31.243670600+05:30" level=warning msg="Error (Unable to complete atomic operation, key
modified) deleting object [endpoint_count 3c8e5677508c285c21382566ceb005c39f127f2f7cdb77caa4f9a51d376f7e08], retrying...."
```

This script is designed to analyze a log file and count the number of occurrences of a specific keyword. It helps in identifying important messages such as errors or warnings within system logs.

The script takes input from the command line. The first argument represents the log file path, and the second optional argument specifies the keyword to search for. If no keyword is provided, the script uses "error" as the default value.

The script first checks whether a file name has been provided and whether the file exists. If the file is not found, it displays an error message and exits. This ensures that the script does not run incorrectly.

Next, a `while` loop is used to read the log file line by line. For each line, the script checks whether the specified keyword is present using the `grep` command. If a match is found, a counter variable is incremented. This allows the script to count how many times the keyword appears in the log file.

After processing the entire file, the script prints the total number of occurrences of the keyword. It also displays the last five matching lines using the `grep` and `tail` commands, which helps in quickly identifying recent issues.

This script demonstrates important shell scripting concepts such as command-line arguments, loops, conditional statements, and text processing. It is useful for system administrators to monitor logs and detect errors efficiently.

Script 5 — The Open Source Manifesto Generator

```
read -p "Tool you use: " TOOL
read -p "Meaning of freedom: " FREEDOM
read -p "What will you build: " BUILD

DATE=$(date)
OUTPUT="manifesto.txt"

echo "On $DATE, I believe that open source means $FREEDOM. Tools like $TOOL show how sharing knowledge empowers everyone. In future, I want to build $BUILD
and share it freely with the world." > $OUTPUT

echo "Saved to $OUTPUT"
cat $OUTPUT
Tool you use: GIT
Meaning of freedom: FREEDOM
What will you build: BUILD
Saved to manifesto.txt
On Thu Mar 26 12:40:37 IST 2026, I believe that open source means FREEDOM. Tools like GIT show how sharing knowledge empowers everyone. In future, I want to
 build BUILD and share it freely with the world.
```

This script is designed to generate a personalized open-source manifesto based on user input. It demonstrates interactive shell scripting by asking the user a series of questions and using their responses to create a meaningful statement.

The script begins by using the `read` command to take input from the user. It asks three questions: the name of an open-source tool the user uses, the meaning of freedom according to the user, and something the user would like to build and share freely. These inputs are stored in variables named `TOOL`, `FREEDOM`, and `BUILD`.

Next, the script captures the current date and time using the `date` command and stores it in a variable. It also defines the name of the output file as `manifesto.txt`.

The script then creates a paragraph by combining the user inputs with a predefined message. This paragraph reflects the user's perspective on open source and its importance. The generated text is written to the output file using the `>` operator, which creates or overwrites the file.

Finally, the script displays a confirmation message and prints the contents of the generated file using the `cat` command.

This script demonstrates key shell scripting concepts such as user input handling, variables, string manipulation, file writing, and output display. It also highlights the philosophy of open source by encouraging users to think about sharing knowledge and contributing to the community.